

oFEC Level 5 / 6 共享译码优化

三种 EarlyStop 组更新、末级 History 修正与跨时刻前瞻调度

16 组 $\times$ 4 code $\cdot$ 8 SISO + 8 HISO

现有 Level 5/6 共享结构



- Level 5 和 Level 6 合计 64 个 code
- 固定分为 16 组, 每组 4 个 code
- 共享资源为 8 路 SISO + 8 路 HISO
- 每次最多使用 8 次 group entry
- EarlyStop 命中行不应继续竞争 HISO/SISO 资源

核心问题:

没有获得普通调度机会的 group, 其中 EarlyStop 命中行是否仍允许更新外信息?

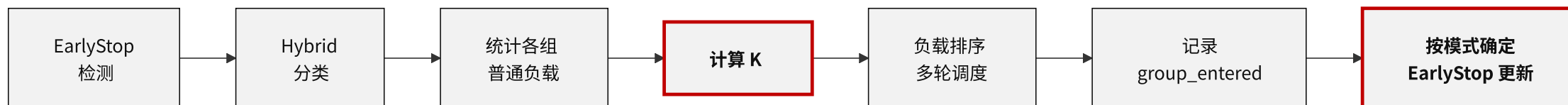
三种方案的总体差异

公共流程和模式差异合并如下：

流程阶段	统一执行内容	三种模式的差异
1. 行级预处理	检测 EarlyStop，并完成 Hybrid 分类	三种模式相同
2. 负载统计	统计 16 个 group 的普通候选，计算非空组数 <code>k</code>	三种模式相同
3. 普通调度	按负载排序，分轮使用最多 8 次 group entry	三种模式相同
4. 进入记录	记录每组是否普通调度进入： <code>group_entered</code>	三种模式相同
5. EarlyStop 更新	命中行决定是否写入新的外信息	AllGroups：所有命中组；EnteredGroupsOnly：仅普通进入组；FillIdleEntries：普通进入组加空闲补位组
6. History 写回	有新外信息则合并；无新外信息则透传当前 prior	三种模式共用修正后的 History 语义

三种模式只改变 EarlyStop 外信息的更新资格，普通 HISO/SISO 的 core 和 MUX 规则不变。

三种模式的公共调度流程

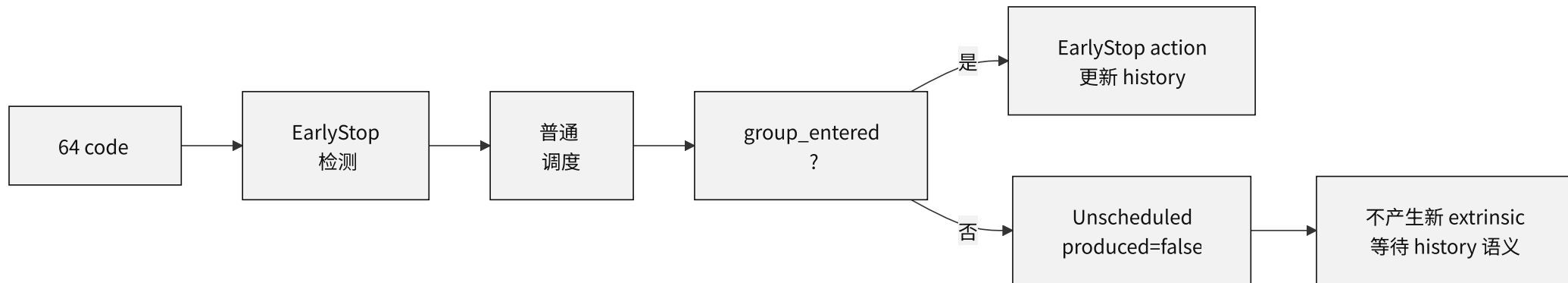


统一流程：

- EarlyStop 检测
- Hybrid 分类
- 统计 16 组普通负载
- 计算 K
- 按负载排序并多轮调度
- 记录 group_entered
- 按模式决定 EarlyStop 是否更新

因此，模式差异集中在最后一个策略判断，而不是前面的 EarlyStop 判决或普通资源调度。

EnteredGroupsOnly: 只更新普通进入组

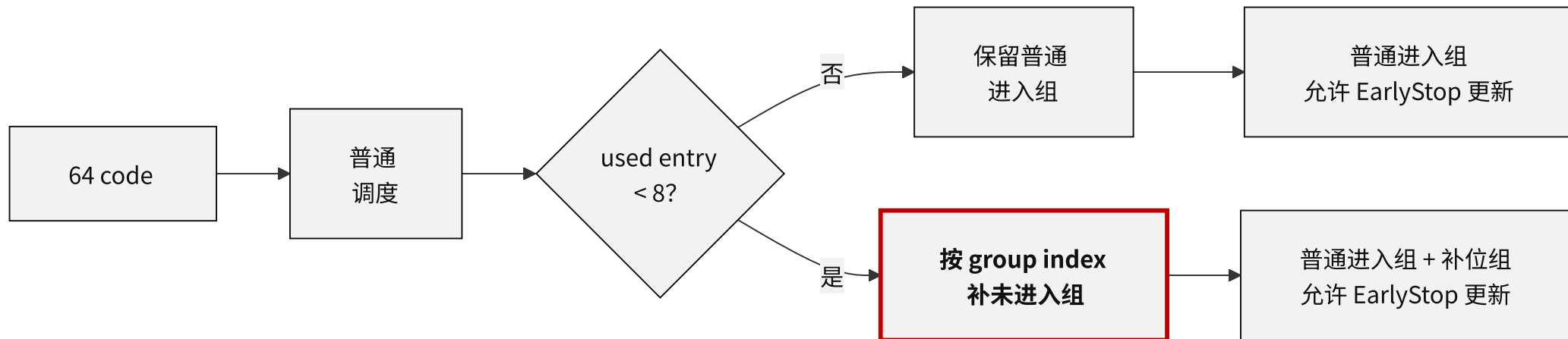


普通调度后:

- group_entered=true: EarlyStop action, 更新 history
- group_entered=false: final_action=Unscheduled, produced=false
- 未进入组不执行 EarlyStop/HISO/SISO, 不产生新 extrinsic

该方案严格遵守没有进入 group 就不能更新的资源语义，但会丢弃未进入组本来可以产生的 EarlyStop 外信息。

FillIdleEntries: 用空闲 entry 补位

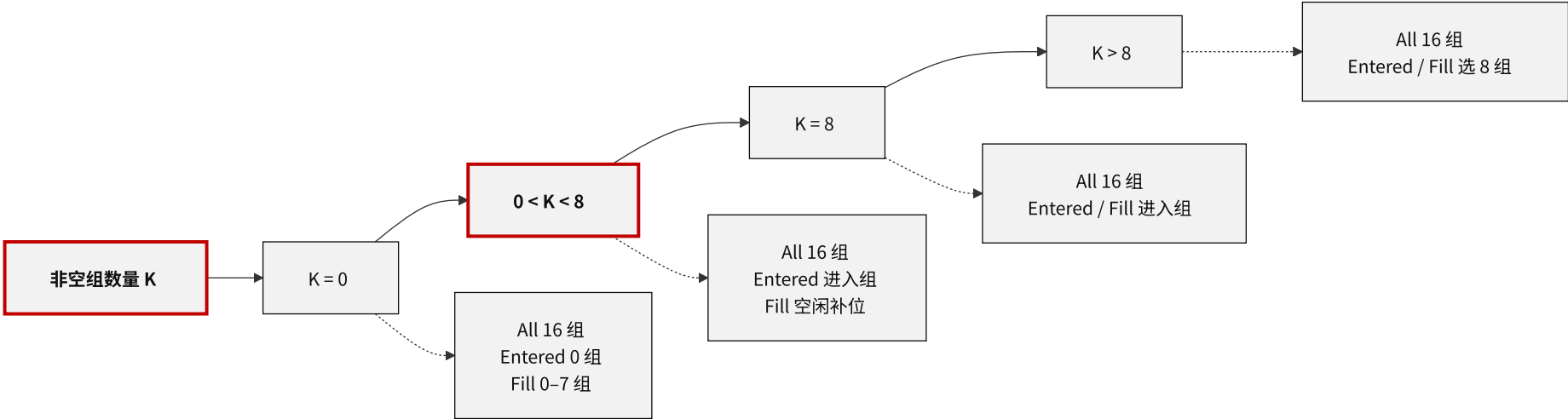


普通调度结束后：

- $\text{remaining} = 8 - \text{used_group_entries}$
- remaining 大于 0 时，按 group index 从小到大选择尚未进入的组
- 补位组不额外运行 HISO/SISO
- 补位组的 EarlyStop 命中行允许更新
- 其他未进入组保持 Unscheduled

它是 EnteredGroupsOnly 的折中方案：利用原本浪费的 group entry，但不放开全部 16 组。

K 的四种调度分支



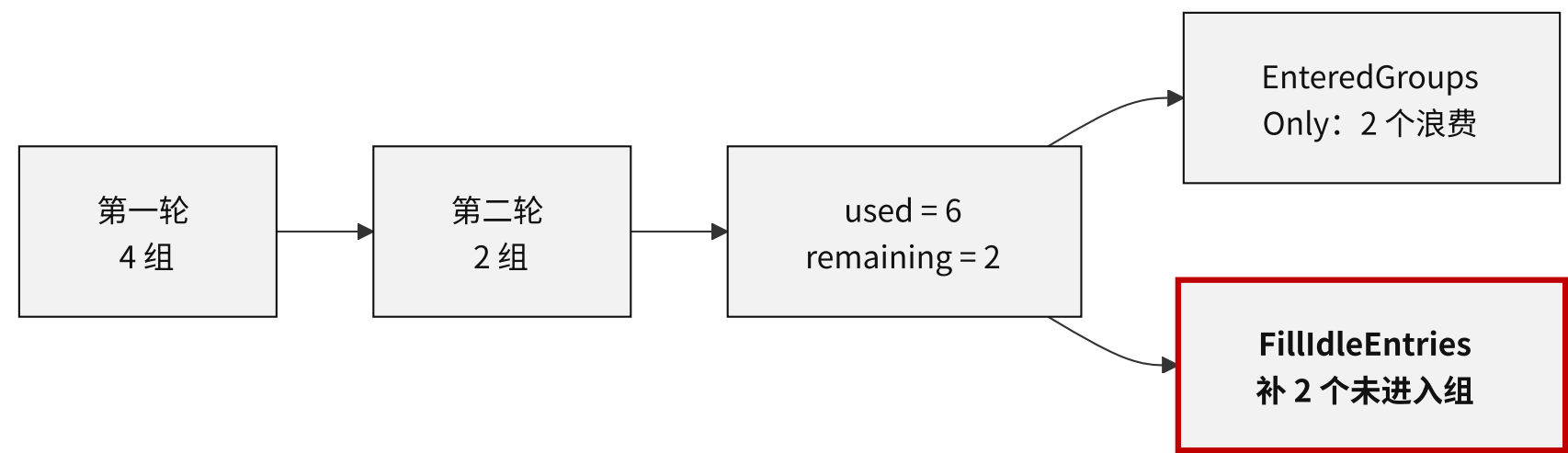
条件	AllGroups	EnteredGroupsOnly	FillIdleEntries
K>8	16 组更新	选中的 8 组更新	与 Entered 相同
K=8	16 组更新	8 个进入组更新	与 Entered 相同
0<K<8	16 组更新	普通进入组更新	使用空闲 entry 补位
K=0	16 组更新	全部不更新	Group 0-7 更新

理论上的 EarlyStop 更新覆盖范围：

$AllGroups \geq FillIdleEntries \geq EnteredGroupsOnly$

这是信息覆盖关系，不保证每一个有限样本的 BER 都严格单调。

空闲 entry 的具体利用方式



示例：第一轮进入 4 个 group，第二轮进入 2 个 group。这里“状态”只描述资源使用情况，“方案”描述三种策略如何处理这一个状态。

类别	名称	动作
调度状态	used=6, remaining=2	普通调度已使用 6 次，还剩 2 次 entry
方案	EnteredGroupsOnly	剩余 2 个 entry 不使用，只更新已进入组
方案	FillIdleEntries	用 2 个空闲 entry 补两个尚未进入的最小 index group
方案	AllGroups	不受 group 是否进入限制，所有 EarlyStop 命中组都可更新

FillIdleEntries 恢复的是部分 EarlyStop 更新机会，不增加 HISO/SISO 解码容量。

性能异常：为什么不像正常调度损失？

最初固定输入下的结果：

模式	Post-FEC errors	Post-FEC BER
AllGroups	10,452	0.000362474
EnteredGroupsOnly	604,882	0.0209773
FillIdleEntries	609,610	0.0211412

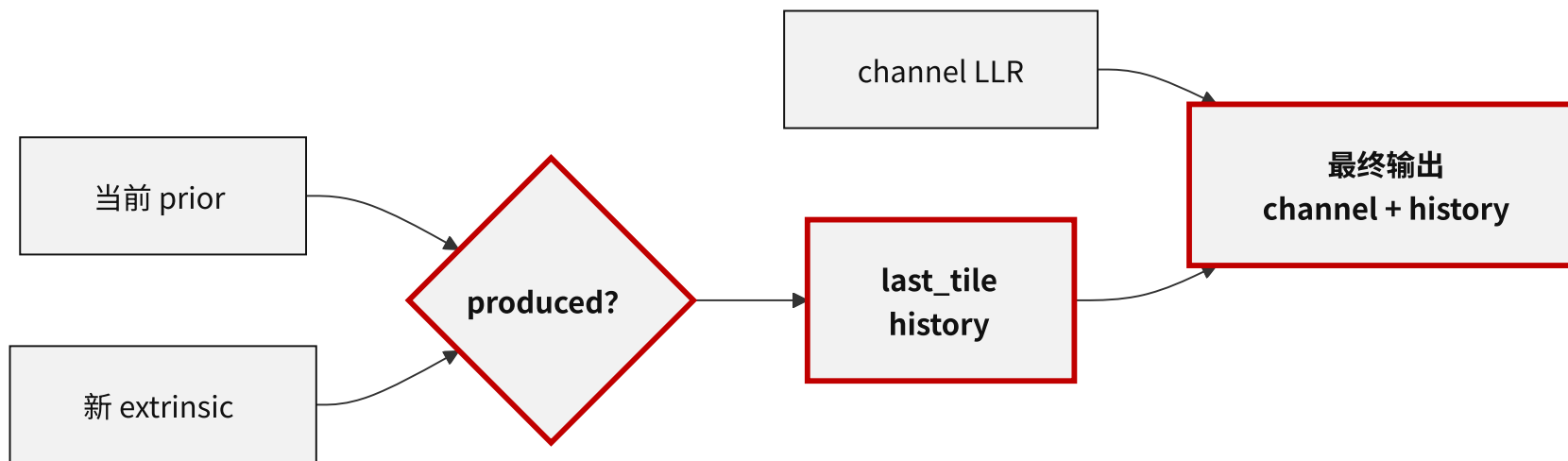
观察：

- 未进入组不更新 EarlyStop，理论上会有性能损失
- 但不应直接退化到接近 Pre-FEC BER：0.0221399
- 后两种模式的异常退化说明最终输出可能丢失了已有软信息

问题从早停更新覆盖不足转向：

最后一个 tile 的 history 是否正确写回？

最终输出中的 History 数据流



这里的 `prior` 指进入当前 tile 之前已经积累的软信息, `extrinsic` 指当前 tile 新计算出的外信息。

最终输出把信道信息和最后一个 tile 的 History 相加:

```
final_output = channel_llr + last_tile_history_llr
```

所以 `last_tile_history_llr` 会直接影响最终的 hard decision。当前 tile 如果产生了新外信息, 写入 `prior + new_extrinsic`; 如果没有产生新外信息, 也必须把当前 `prior` 写回, 不能让旧窗口值或 0 覆盖它。

根因：produced=false 时旧 History 被保留

这里 `produced` 表示“当前行是否真正产生了新的译码输出”，不是“这一行是否存在”。

修复前：

- `produced=true`: `history=prior+new_extrinsic`
- `produced=false`: 不写 `history`, 保留旧窗口值或初始化 0

错误输出: `final_output_old = channel + stale_or_zero_history`

正确应为: `final_output = channel + 当前 prior`

因此, `produced=false` 的含义是“本行不新增信息”，而不是“本行的已有信息清零”。受影响范围包括资源不足的 `Unscheduled` 行, 以及未获得 `EarlyStop` 更新资格的命中行。

根因结论: 不是 `EarlyStop` 判决本身错误, 而是 `produced=false` 时没有把当前 `prior` 透传到最后一个 `tile` 的 `History`。

History 修正：透传当前 Prior



修正后的语义：

- `produced=true`: `history=prior+new_extrinsic`
- `produced=false`: `history=prior`

换句话说，`produced=false` 只禁止新增译码信息，不禁止保存已有的 `prior`。

明确保持不变：

- `Unscheduled` 仍不执行 `EarlyStop`、`HISO`、`SISO`
- `produced` 仍为 `false`
- `tile_out` 仍然不修改
- 只修正最后 tile 的 `last_tile_history_accum`

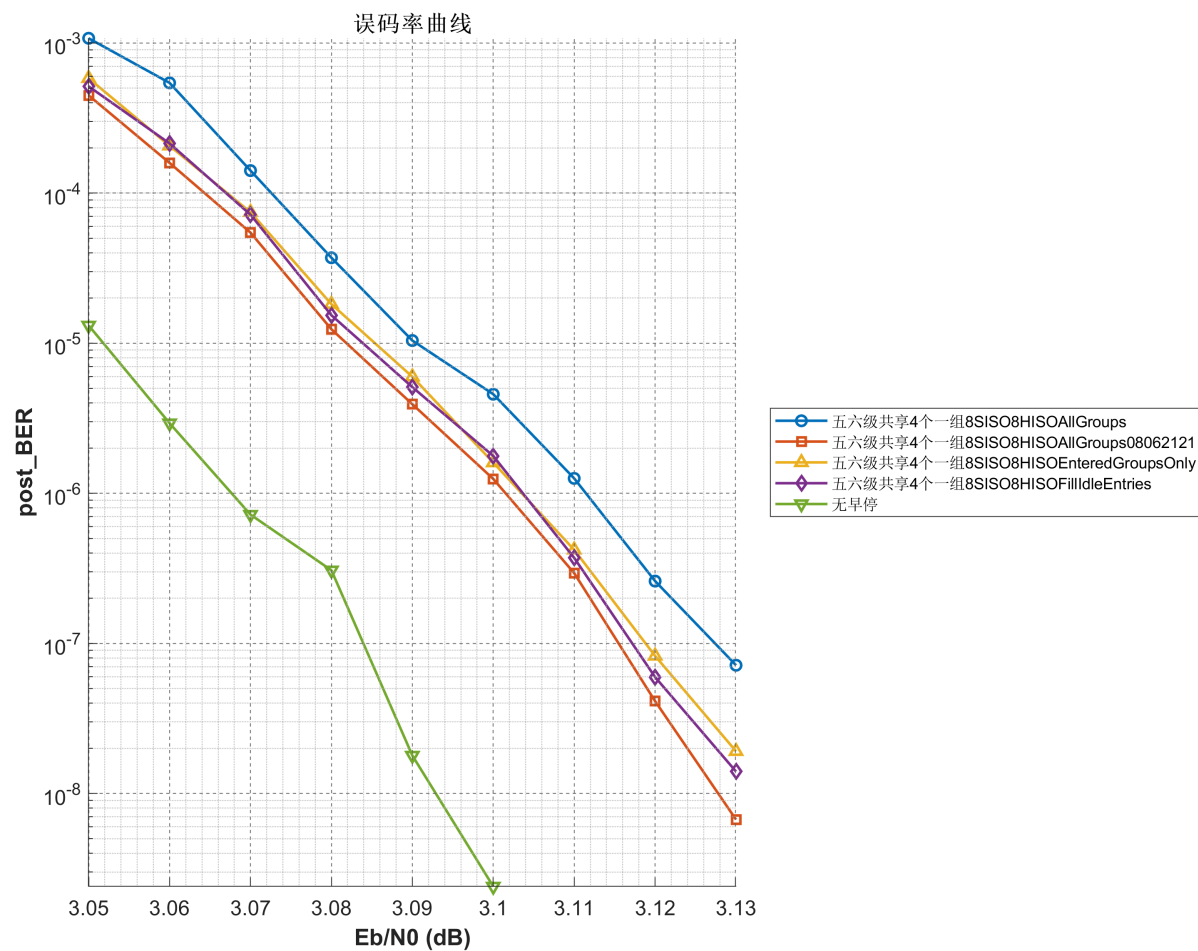
修复有效性: AllGroups 前后对照

项目	修复前	修复后
Post-FEC errors	10,452	3,482
Post-FEC BER	0.000362474	0.000120755
调度统计	相同	相同
Level 5 Unscheduled	4,750	4,750
Level 6 Unscheduled	3,367	3,367

结论:

- 错误数下降 66.7%
- EarlyStop 命中、HISO/SISO 分配和 Unscheduled 数量均未改变
- 性能改善来自 produced=false 行的 prior 透传

History 修正前后的 BER 曲线



修正后性能对比与结论

修正后的三种方案数据来自 OneDrive 目录中的三份 CSV。

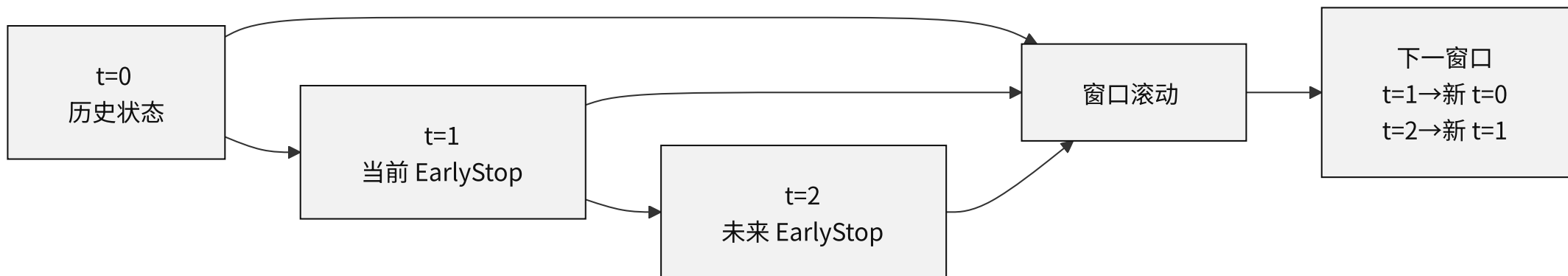
- 修正后 AllGroups 曲线在 3.05–3.13 dB 范围内最低
- FillIdleEntries 整体优于 EnteredGroupsOnly
- FillIdleEntries 利用空闲 entry 恢复了部分 EarlyStop 外信息
- 三种方案的剩余差异才代表真正的调度策略差异

理论和实践的总体排序：

AllGroups > FillIdleEntries > EnteredGroupsOnly

这个排序代表整体趋势，不要求每个单点都严格单调。

新方案：从单时刻扩展到三时刻观察



- 当前：一次观察 1 批 64 code
- 新方案：一次观察 $t=0$ 、 $t=1$ 、 $t=2$ ，共 192 个 code

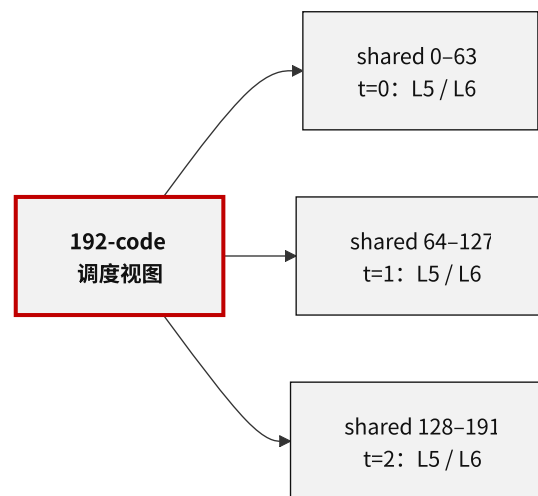
信息分工：

- $t=0$ ：已经处理过，提供完整解码状态
- $t=1$ ：当前批次，提供 EarlyStop 信息
- $t=2$ ：未来批次，提供 EarlyStop 前瞻信息

目标：

用历史遗留、当前负载和未来负载共同决定当前资源优先级。

192-code 视图与固定索引

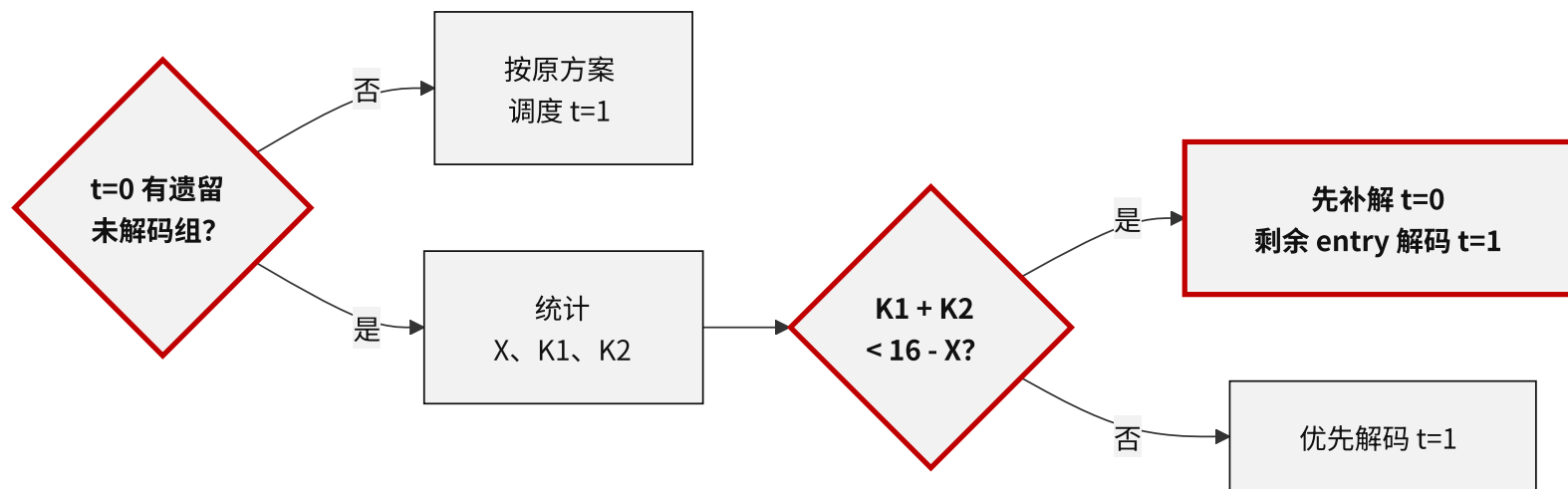


固定布局:

- shared[0..63]: t=0, Level 5 为 0..31, Level 6 为 32..63
- shared[64..127]: t=1, Level 5 为 64..95, Level 6 为 96..127
- shared[128..191]: t=2, Level 5 为 128..159, Level 6 为 160..191

调度统计量: X 为 t=0 遗留未解码组数; K1/K2 分别为 t=1/t=2 普通非空组数。

跨时刻调度决策



核心规则:

- t=0 无遗留 code: 按原方案调度 t=1
- t=0 有遗留 code 且 $K1 + K2 < 16 - X$: 先补解 t=0, 剩余 entry 再解码 t=1
- $K1 + K2 \geq 16 - X$: 优先解码 t=1

边界约束:

- t=2 只提供前瞻信息, 本次不直接解码
- 每次当前译码仍最多使用 8 次 group entry
- 窗口滚动后重新统计 X/K1/K2